

# 05 — State-Space GPs and the Kalman Duality

**What you'll learn:** why a temporal GP with a Markovian kernel is exactly a Kalman filter/smoothing, giving  $O(n)$  inference and free interpolation, and how this connects to the repo's imu-drift crate.

**Prereqs:** 01-03; a passing acquaintance with Kalman filters helps (04 §1).

**Why it matters here:** this is the route from the demo GP ( $O(n^3)$ ) to an edge GP ( $O(n)$ ), and it reuses physics we already have.

## 1. The big idea

For **temporal** GPs (input is time) with certain kernels, the  $O(n^3)$  cost of doc 03 **vanishes**. The reason: those kernels correspond to a **linear stochastic differential equation (SDE)** driven by white noise. A linear SDE is a **Markov** process — the future depends on the present, not the whole past — and Bayesian inference on a linear-Gaussian Markov process is exactly the **Kalman filter** (forward) and **RTS smoother** (backward). Both are  **$O(n)$**  in the number of timesteps.

|               |   |                                |   |                              |
|---------------|---|--------------------------------|---|------------------------------|
| Temporal GP   | ≡ | linear SDE (state-space model) | ≡ | Kalman filter / RTS smoother |
| (kernel view) |   | (dynamics view)                |   | (algorithm view)             |

Same posterior mean and variance as the batch GP — just computed in linear time by a recursion instead of a cubic matrix factorization.

## 2. How a kernel becomes a state-space model

You augment the scalar function  $f(t)$  into a small **state vector**  $s(t) = [f, \dot{f}, \ddot{f}, \dots]$  (the function and a few derivatives), evolving as:

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| $ds(t) = F s(t) dt + L d\beta(t)$ | (linear SDE)                        |
| $f(t) = H s(t)$                   | (we observe the function component) |

$F, L, H$  are small constant matrices determined by the kernel;  $d\beta$  is white noise. Discretizing over the (possibly irregular) time steps yields the familiar Kalman predict/update matrices.

- **Matérn kernels are exactly representable.** Matérn-1/2  $\rightarrow$  a 1-state SDE (Ornstein-Uhlenbeck);

Matérn-3/2  $\rightarrow$  2 states; Matérn-5/2  $\rightarrow$  3 states. The half-integer  $\nu$  is precisely what makes the spectral density rational and the state finite. This is a second, decisive reason we chose Matérn

in doc 02. (RBF needs an infinite state and is only approximated in state-space form.)

## 3. Why it's $O(n)$ : the precision matrix is block-tridiagonal

The batch GP works with the dense covariance  $K$ . The state-space view works with its inverse, the

**precision matrix**  $K^{-1}$ , which for a Markov process is **block-tridiagonal** — each timestep only couples to its immediate neighbours. Solving a block-tridiagonal system is  **$O(n)$** , not  $O(n^3)$ . This is the same structure exploited by Barfoot, Tong & Anderson's *Trajectory Estimation as Exactly Sparse GP Regression* (arXiv:1412.0630): they pose continuous-time robot trajectory estimation as a GP with a "white-noise-on-acceleration" (constant-velocity) prior, note the block-tridiagonal precision, and solve it in linear time.

## 4. Free, principled interpolation at any time

Because the model is continuous-time, you can query the posterior at **any** timestamp, not just where you have measurements — with closed-form mean and covariance ("GP interpolation"). This is perfect for our asynchronous sensors: query the velocity GP exactly at an IMU step, or at an expected DVL tick during a dropout. The repo already does linear interpolation of IMU between samples (`synchronization.py _interpolate_imu`); the state-space GP generalizes that to a probabilistic, dynamics-aware interpolation with uncertainty.

## 5. The connection to imu-drift (we already own the prior)

The Rust `imu-drift` crate (`imu-drift/src/processes.rs`) provides closed-form drift means and variances for inertial error processes. These are precisely the marginal statistics of the state-space GP priors:

| <code>`imu-drift` process</code>            | Variance law                                    | State-space GP equivalent                                                                         |
|---------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>`IntegratedVelocityRandomWalk`</code> | $\sigma^2(t) = N \cdot t^3/3$                   | position under a <b>white-noise-on-acceleration</b> (constant-velocity) prior — the Barfoot prior |
| <code>`VelocityRandomWalk`</code>           | $\sigma^2(t) = N \cdot t$                       | velocity under white-noise-on-acceleration (Wiener)                                               |
| <code>`GaussMarkov`</code> (OU)             | $\sigma^2(t) = \sigma_b^2(1 - e^{-2t/\tau})$    | <b>Matérn-1/2</b> kernel (exponential), 1-state SDE                                               |
| <code>`ConstantGyroBias`, biases</code>     | linear/quadratic mean terms                     | deterministic drift / random-constant states                                                      |
| <code>`GyroBiasGravityCoupling`</code>      | mean $\propto \frac{1}{2}g \cdot b_g \cdot t^3$ | cubic cross-coupling term in the augmented state                                                  |

**Implication:** the physically-calibrated process-noise parameters in `imu-drift` (from Allan-variance analysis, `imu-drift/src/allan.rs`) can be dropped straight into a state-space GP as its

F/L/Q — giving an edge-deployable GP whose prior is grounded in the vehicle's actual IMU characterization, not guessed hyperparameters. This is the heart of the edge roadmap (`implementation/04`).

## 6. Demo vs edge — same interface

|               | Demo build (now)                      | Edge build (later)                             |
|---------------|---------------------------------------|------------------------------------------------|
| Engine        | GPyTorch batch GP on a sliding window | state-space GP = Kalman/RTS recursion          |
| Cost          | $O(n^3)$ but $n$ is tiny (windowed)   | <b><math>O(n)</math></b> , constant memory     |
| Kernel source | learned hyperparameters               | <code>`imu-drift`</code> Allan-variance params |
| Deps          | torch/gpytorch                        | numpy only (or <code>no_std</code> Rust)       |

Interface | `ingest()/fit()/predict()` | **\*\*identical\*\***

Because both hide behind the same `predict()/ingest()` API (implementation/02), swapping the engine for edge is an internal change — the pipeline wiring is untouched.

## 7. Going multi-dimensional / spatio-temporal

The same idea extends to space-time: keep the Markov structure in time (Kalman) and use a spatial basis at each step → **infinite-dimensional Kalman filtering** (Särkkä, Solin, Hartikainen). This is the algorithmic cousin of the SPDE/GMRF approach in doc 06 and the bridge to modelling an ocean-current field efficiently.

### Key takeaways

- A temporal GP with a **Matérn** (Markovian) kernel **is** a Kalman filter/RTS smoother →  **$O(n)$**  inference, same posterior as the batch GP.
- The precision matrix is **block-tridiagonal**; Barfoot et al. use exactly this for continuous-time trajectory estimation.
- You get **closed-form interpolation at any time** — ideal for asynchronous DVL/IMU and for bridging.
- imu-drift's closed-form variances **are** these GP priors; its Allan-variance params can seed an edge-deployable, physically-grounded state-space GP.

### Further reading

- Särkkä & Solin, Applied Stochastic Differential Equations (free) — kernels ↔ SDEs.
- Barfoot, Tong, Anderson, arXiv:1412.0630 — exactly-sparse GP trajectory estimation.
- Hartikainen & Särkkä, Kalman filtering and smoothing solutions to temporal GP regression.
- Next: 06 — The SPDE approach (the PDE duality).